DBMS MINI PROJECT

AI-Driven Cyber Threat Intelligence & Attack Campaign Correlation System

SQL IMPLEMENTATION AND EXECUTION REPORT

Name : AMIT

SRN:PES2UG24CS624

NAME: M SANGAMESH PRASAD

SRN:PES2UG24CS640

1. SQL Implementation Overview

This report presents the completed SQL implementation and execution evidence for the AI-Driven Cyber Threat Intelligence & Attack Campaign Correlation System. The implementation demonstrates project data, entity relationships, multi-table queries, aggregate functions, subqueries, views, indexes and transaction management using PostgreSQL.

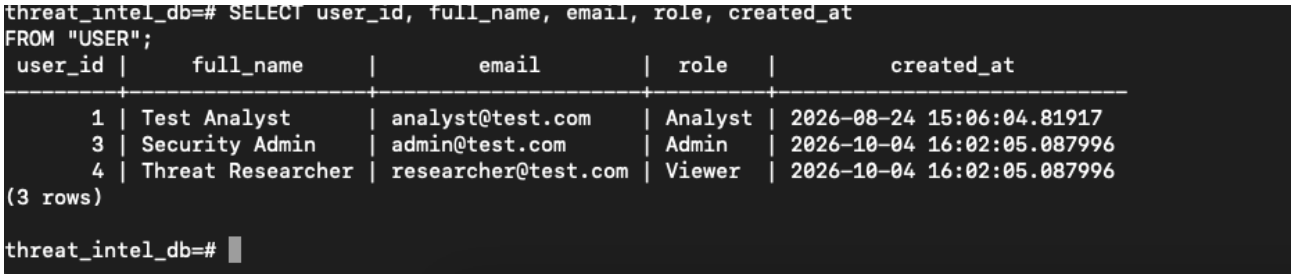
2. SQL Concepts Demonstrated

- Data insertion and retrieval using INSERT and SELECT
- Multi-table JOIN operations
- UPDATE operations
- Aggregate functions with GROUP BY and HAVING
- Subqueries
- CASE expressions
- Database VIEW creation and retrieval
- EXISTS and NOT EXISTS
- Indexes
- Transactions using BEGIN, SAVEPOINT, ROLLBACK TO SAVEPOINT and COMMIT
- Full transaction ROLLBACK
- Project-specific threat intelligence correlation queries

3. SQL Execution Evidence

3.1 Base Project Data

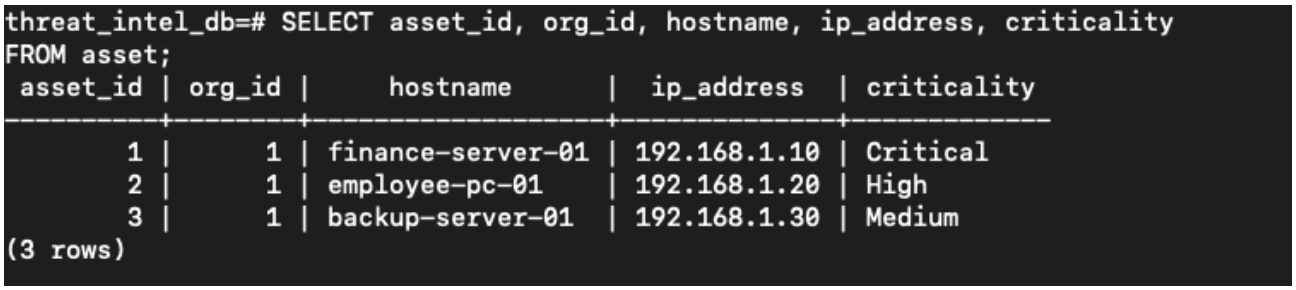
Figure 1. USER table – user records



```
threat_intel_db=# SELECT user_id, full_name, email, role, created_at
FROM "USER";
 user_id |     full_name     |        email        |  role   |         created_at
---------+-------------------+---------------------+---------+----------------------------
       1 | Test Analyst      | analyst@test.com    | Analyst | 2026-08-24 15:06:04.81917
       3 | Security Admin    | admin@test.com      | Admin   | 2026-10-04 16:02:05.087996
       4 | Threat Researcher | researcher@test.com | Viewer  | 2026-10-04 16:02:05.087996
(3 rows)

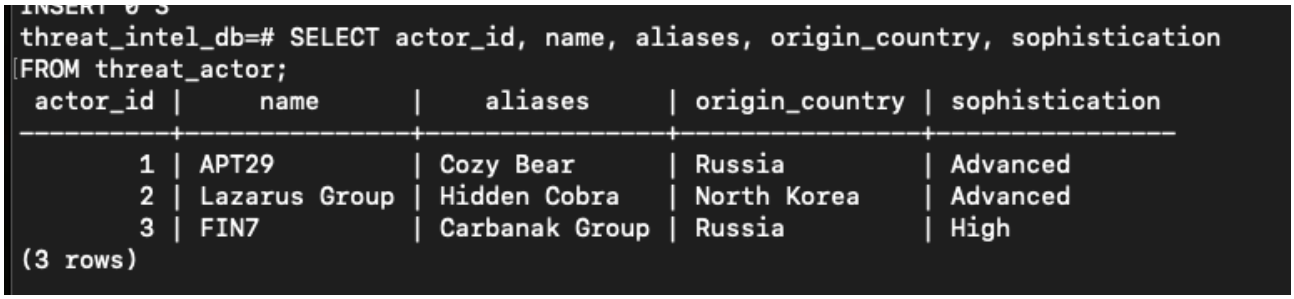
threat_intel_db=#
```

Figure 2. Asset table – organizational assets



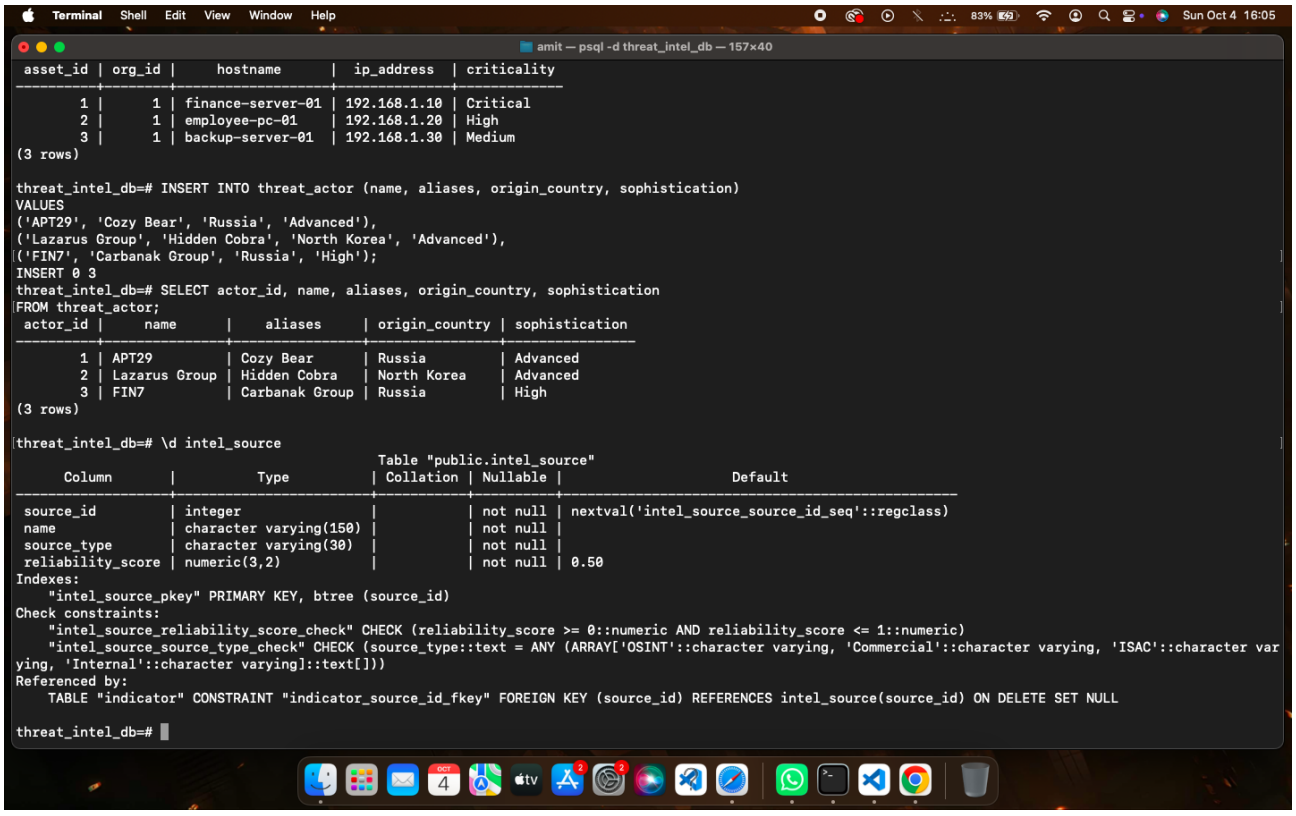
```
threat_intel_db=# SELECT asset_id, org_id, hostname, ip_address, criticality
FROM asset;
 asset_id | org_id |     hostname      |  ip_address  | criticality
----------+--------+-------------------+--------------+-------------
        1 |      1 | finance-server-01 | 192.168.1.10 | Critical
        2 |      1 | employee-pc-01    | 192.168.1.20 | High
        3 |      1 | backup-server-01  | 192.168.1.30 | Medium
(3 rows)
```

Figure 3. Threat Actor table – threat actor records



```
INSERT 0 3
threat_intel_db=# SELECT actor_id, name, aliases, origin_country, sophistication
FROM threat_actor;
 actor_id |      name       |     aliases      | origin_country | sophistication
----------+-----------------+------------------+----------------+----------------
        1 | APT29           | Cozy Bear        | Russia         | Advanced
        2 | Lazarus Group   | Hidden Cobra     | North Korea    | Advanced
        3 | FIN7            | Carbanak Group   | Russia         | High
(3 rows)
```

Figure 4. Intelligence Source table – source records



```
Terminal  Shell  Edit  View  Window  Help                                   Sun Oct 4 16:05
amit — psql -d threat_intel_db — 157×40
 asset_id | org_id |     hostname      |  ip_address  | criticality
----------+--------+-------------------+--------------+-------------
        1 |      1 | finance-server-01 | 192.168.1.10 | Critical
        2 |      1 | employee-pc-01    | 192.168.1.20 | High
        3 |      1 | backup-server-01  | 192.168.1.30 | Medium
(3 rows)

threat_intel_db=# INSERT INTO threat_actor (name, aliases, origin_country, sophistication)
VALUES
('APT29', 'Cozy Bear', 'Russia', 'Advanced'),
('Lazarus Group', 'Hidden Cobra', 'North Korea', 'Advanced'),
('FIN7', 'Carbanak Group', 'Russia', 'High');
INSERT 0 3
threat_intel_db=# SELECT actor_id, name, aliases, origin_country, sophistication
FROM threat_actor;
 actor_id |     name      |    aliases     | origin_country | sophistication
----------+---------------+----------------+----------------+----------------
        1 | APT29         | Cozy Bear      | Russia         | Advanced
        2 | Lazarus Group | Hidden Cobra   | North Korea    | Advanced
        3 | FIN7          | Carbanak Group | Russia         | High
(3 rows)

threat_intel_db=# \d intel_source
                                      Table "public.intel_source"
      Column       |          Type          | Collation | Nullable |                      Default
-------------------+------------------------+-----------+----------+-----------------------------------------------
 source_id         | integer                |           | not null | nextval('intel_source_source_id_seq'::regclass)
 name              | character varying(150) |           | not null |
 source_type       | character varying(30)  |           | not null |
 reliability_score | numeric(3,2)           |           | not null | 0.50
Indexes:
    "intel_source_pkey" PRIMARY KEY, btree (source_id)
Check constraints:
    "intel_source_reliability_score_check" CHECK (reliability_score >= 0::numeric AND reliability_score <= 1::numeric)
    "intel_source_source_type_check" CHECK (source_type::text = ANY (ARRAY['OSINT'::character varying, 'Commercial'::character varying, 'ISAC'::character var
ying, 'Internal'::character varying]::text[]))
Referenced by:
    TABLE "indicator" CONSTRAINT "indicator_source_id_fkey" FOREIGN KEY (source_id) REFERENCES intel_source(source_id) ON DELETE SET NULL

threat_intel_db=#
```

Figure 5. Indicator table – indicators / IOCs

```
threat_intel_db=# SELECT source_id, name, source_type, reliability_score
FROM intel_source;
 source_id |      name       | source_type | reliability_score
-----------+-----------------+-------------+-------------------
         1 | AlienVault OTX  | OSINT       |              0.90
         2 | VirusTotal      | Commercial  |              0.95
         3 | Internal SOC    | Internal    |              0.85
(3 rows)
```

Figure 6. Campaign table – attack campaigns

```
threat_intel_db=# SELECT indicator_id,
       source_id,
       ioc_type,
       value,
       confidence_score,
       first_seen,
       last_seen
FROM indicator
ORDER BY indicator_id;
 indicator_id | source_id | ioc_type |                                value                                | confidence_score | first_seen | last_seen
--------------+-----------+----------+---------------------------------------------------------------------+------------------+------------+------------
            1 |         1 | IP       | 10.10.10.25                                                         |             0.92 | 2026-01-10 | 2026-09-20
            2 |         2 | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |             0.98 | 2026-02-15 | 2026-09-22
            3 |         1 | Domain   | malicious-example.com                                               |             0.88 | 2026-03-01 | 2026-09-25
            4 |         3 | IP       | 192.168.10.50                                                       |             0.76 | 2026-04-05 |
            5 |         2 | URL      | http://malicious-example.com/login                                  |             0.81 | 2026-05-10 | 2026-09-30
(5 rows)
```

Figure 7. TTP table – MITRE techniques

```
threat_intel_db=# SELECT
    campaign_id,
    actor_id,
    name,
    objective,
    start_date,
    end_date,
    status
FROM campaign
ORDER BY campaign_id;
 campaign_id | actor_id |        name        |             objective              | start_date |  end_date  | status
-------------+----------+--------------------+------------------------------------+------------+------------+---------
           1 |        1 | Operation Shadow   | Credential theft and data collection | 2026-01-01 |            | Active
           2 |        2 | Operation Lazarus  | Financially motivated intrusion    | 2026-02-01 | 2026-08-31 | Closed
           3 |        3 | Operation Carbanak | Banking infrastructure compromise  | 2026-03-01 |            | Dormant
(3 rows)
```

Figure 8. Malware Sample table – malware records

```
threat_intel_db=# SELECT ttp_id,
       mitre_technique_id,
       tactic,
       name
FROM ttp
ORDER BY ttp_id;
 ttp_id | mitre_technique_id |        tactic        |                name
--------+--------------------+----------------------+-------------------------------------
      1 | T1059              | Execution            | Command and Scripting Interpreter
      2 | T1566              | Initial Access       | Phishing
      3 | T1055              | Privilege Escalation | Process Injection
      4 | T1071.001          | Command and Control  | Web Protocols
(4 rows)
```

Figure 9. Vulnerability table – CVE records

```
threat_intel_db=# SELECT malware_id,
       sha256_hash,
       family_name,
       file_type,
       first_seen
FROM malware_sample
ORDER BY malware_id;
 malware_id |                            sha256_hash                             |  family_name  | file_type | first_seen
------------+--------------------------------------------------------------------+---------------+-----------+------------
          1 | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa | Emotet        | PE32      | 2026-10-04
          2 | bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb | WannaCry      | PE32      | 2026-10-04
          3 | ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc | Cobalt Strike | DLL       | 2026-10-04
(3 rows)
```

3.2 Relationship and JOIN Queries

Figure 10. Malware–Vulnerability JOIN

```
threat_intel_db=# SELECT
    m.malware_id,
    m.family_name AS malware,
    v.cve_id,
    v.cvss_score,
    v.description
FROM malware_vuln mv
JOIN malware_sample m
    ON mv.malware_id = m.malware_id
JOIN vulnerability v
    ON mv.cve_id = v.cve_id
ORDER BY m.malware_id;
 malware_id |    malware    |     cve_id     | cvss_score |                        description
------------+---------------+----------------+------------+-----------------------------------------------------------
          1 | Emotet        | CVE-2021-44228 |       10.0 | Remote code execution vulnerability in Log4j.
          2 | WannaCry      | CVE-2017-0144  |        9.8 | Critical vulnerability associated with SMBv1.
          3 | Cobalt Strike | CVE-2019-0708  |        9.8 | Remote Desktop Services remote code execution vulnerability.
(3 rows)
```

Figure 11. Campaign–Indicator JOIN

```
threat_intel_db=# SELECT
    c.name AS campaign,
    i.ioc_type,
    i.value AS indicator,
    i.confidence_score,
    ci.correlation_score
FROM campaign_indicator ci
JOIN campaign c
    ON ci.campaign_id = c.campaign_id
JOIN indicator i
    ON ci.indicator_id = i.indicator_id
ORDER BY ci.correlation_score DESC;
      campaign      | ioc_type |                                                                  indicator                                                                   | confidence_score | correlation_score
--------------------+----------+----------------------------------------------------------------------------------------------------------------------------------------------+------------------+-------------------
 Operation Shadow   | IP       | 10.10.10.25                                                                                                                                  |             0.92 |             0.950
 Operation Shadow   | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |             0.98 |             0.910
 Operation Lazarus  | Domain   | malicious-example.com                                                                                                                        |             0.88 |             0.880
 Operation Carbanak | URL      | http://malicious-example.com/login                                                                                                           |             0.81 |             0.840
 Operation Carbanak | IP       | 192.168.10.50                                                                                                                                |             0.76 |             0.790
(5 rows)
```

Figure 12. Campaign–TTP JOIN

```
threat_intel_db=# SELECT
    c.name AS campaign,
    t.mitre_technique_id,
    t.tactic,
    t.name AS technique,
    ct.notes
FROM campaign_ttp ct
JOIN campaign c
    ON ct.campaign_id = c.campaign_id
JOIN ttp t
    ON ct.ttp_id = t.ttp_id
ORDER BY c.campaign_id;
      campaign      | mitre_technique_id |       tactic        |            technique             |                        notes
--------------------+--------------------+---------------------+----------------------------------+-------------------------------------------------------
 Operation Shadow   | T1059              | Execution           | Command and Scripting Interpreter | Command execution observed after initial compromise
 Operation Shadow   | T1566              | Initial Access      | Phishing                         | Phishing used for initial access
 Operation Lazarus  | T1055              | Privilege Escalation | Process Injection               | Process injection observed
 Operation Carbanak | T1071.001          | Command and Control | Web Protocols                    | HTTP-based command and control observed
(4 rows)
```

Figure 13. Indicator–Malware JOIN

```
threat_intel_db=# SELECT
    i.indicator_id,
    i.ioc_type,
    i.value AS indicator,
    m.malware_id,
    m.family_name AS malware,
    m.file_type
FROM indicator_malware im
JOIN indicator i
    ON im.indicator_id = i.indicator_id
JOIN malware_sample m
    ON im.malware_id = m.malware_id
ORDER BY i.indicator_id;
 indicator_id | ioc_type |                                                                  indicator                                                                   | malware_id |    malware    | file_type
--------------+----------+----------------------------------------------------------------------------------------------------------------------------------------------+------------+---------------+-----------
            2 | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |          1 | Emotet        | PE32
            3 | Domain   | malicious-example.com                                                                                                                        |          2 | WannaCry      | PE32
            5 | URL      | http://malicious-example.com/login                                                                                                           |          3 | Cobalt Strike | DLL
(3 rows)
```

Figure 14. Alert–Asset–Indicator–User JOIN

```
INSERT 0 4
threat_intel_db=# SELECT
    al.alert_id,
    a.hostname,
    a.ip_address,
    i.ioc_type,
    i.value AS indicator,
    u.full_name AS analyst,
    al.severity,
    al.status,
    al.detected_at
FROM alert al
JOIN asset a
    ON al.asset_id = a.asset_id
JOIN indicator i
    ON al.indicator_id = i.indicator_id
LEFT JOIN "USER" u
    ON al.user_id = u.user_id
ORDER BY al.alert_id;
 alert_id |     hostname      |  ip_address  | ioc_type |                          indicator                           |      analyst      | severity |
  status     |        detected_at
----------+-------------------+--------------+----------+--------------------------------------------------------------+-------------------+----------+--
------------+----------------------------
        9 | finance-server-01 | 192.168.1.10 | IP       | 10.10.10.25                                                  | Test Analyst      | High     | O
pen        | 2026-10-04 16:17:39.339226
       10 | finance-server-01 | 192.168.1.10 | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa | Test Analyst      | Critical | I
nvestigating | 2026-10-04 16:17:39.339226
       11 | employee-pc-01    | 192.168.1.20 | Domain   | malicious-example.com                                        | Security Admin    | Medium   | R
esolved    | 2026-10-04 16:17:39.339226
       12 | employee-pc-01    | 192.168.1.20 | URL      | http://malicious-example.com/login                           | Threat Researcher | High     | O
pen        | 2026-10-04 16:17:39.339226
(4 rows)
```

Figure 15. Report–User–Campaign JOIN

```
threat_intel_db=# SELECT
    r.report_id,
    r.title,
    u.full_name AS author,
    c.name AS campaign,
    r.classification,
    r.published_date
FROM report r
JOIN "USER" u
    ON r.user_id = u.user_id
LEFT JOIN campaign c
    ON r.campaign_id = c.campaign_id
ORDER BY r.report_id;
 report_id |          title           |      author       |      campaign      | classification | published_date
-----------+--------------------------+-------------------+--------------------+----------------+----------------
         1 | Operation Shadow Analysis | Test Analyst      | Operation Shadow   | TLP:AMBER      | 2026-08-01
         2 | Lazarus Campaign Report   | Security Admin    | Operation Lazarus  | TLP:GREEN      | 2026-08-15
         3 | Carbanak Investigation    | Threat Researcher | Operation Carbanak | TLP:RED        | 2026-09-01
(3 rows)
```

3.3 Data Manipulation and Aggregate Queries

Figure 16. UPDATE operation

```
threat_intel_db=# UPDATE alert
SET status = 'Investigating'
WHERE asset_id = 1
  AND indicator_id = 1;
UPDATE 1
threat_intel_db=# SELECT
    alert_id,
    asset_id,
    indicator_id,
    severity,
    status
FROM alert
WHERE asset_id = 1
  AND indicator_id = 1;
 alert_id | asset_id | indicator_id | severity |    status
----------+----------+--------------+----------+---------------
        9 |        1 |            1 | High     | Investigating
(1 row)
```

Figure 17. GROUP BY with COUNT

```
threat_intel_db=# SELECT
    severity,
    COUNT(*) AS alert_count
FROM alert
GROUP BY severity
ORDER BY alert_count DESC;
  severity | alert_count
-----------+-------------
 High      |           2
 Medium    |           1
 Critical  |           1
(3 rows)
```

Figure 18. GROUP BY with HAVING

```
threat_intel_db=# SELECT
    c.name AS campaign,
    COUNT(ci.indicator_id) AS indicator_count
FROM campaign c
JOIN campaign_indicator ci
    ON c.campaign_id = ci.campaign_id
GROUP BY c.campaign_id, c.name
HAVING COUNT(ci.indicator_id) > 1;
       campaign       | indicator_count
----------------------+-----------------
 Operation Carbanak   |               2
 Operation Shadow     |               2
(2 rows)
```

Figure 19. AVG with GROUP BY

```
threat_intel_db=# SELECT
    ioc_type,
    ROUND(AVG(confidence_score), 2) AS average_confidence
FROM indicator
GROUP BY ioc_type
ORDER BY average_confidence DESC;
 ioc_type | average_confidence
----------+--------------------
 Hash     |               0.98
 Domain   |               0.88
 IP       |               0.84
 URL      |               0.81
(4 rows)
```

3.4 Advanced SQL Queries

Figure 20. Subquery – indicators above average confidence

```
threat_intel_db=# SELECT
    indicator_id,
    ioc_type,
    value,
    confidence_score
FROM indicator
WHERE confidence_score >
      (SELECT AVG(confidence_score)
       FROM indicator)
ORDER BY confidence_score DESC;
 indicator_id | ioc_type |                                  value                                  | confidence_score
--------------+----------+-------------------------------------------------------------------------+------------------
            2 | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |             0.98
            1 | IP       | 10.10.10.25                                                             |             0.92
            3 | Domain   | malicious-example.com                                                   |             0.88
(3 rows)
```

Figure 21. Subquery – campaigns above average correlation

```
threat_intel_db=# SELECT
    c.name AS campaign,
    ci.correlation_score
FROM campaign c
JOIN campaign_indicator ci
    ON c.campaign_id = ci.campaign_id
WHERE ci.correlation_score >
      (
          SELECT AVG(correlation_score)
          FROM campaign_indicator
      )
ORDER BY ci.correlation_score DESC;
     campaign      | correlation_score
-------------------+-------------------
 Operation Shadow  |             0.950
 Operation Shadow  |             0.910
 Operation Lazarus |             0.880
(3 rows)
```

Figure 22. CASE expression – alert priority classification

```
threat_intel_db=# SELECT
    alert_id,
    severity,
    status,
    CASE
        WHEN severity = 'Critical' THEN 'Immediate Action'
        WHEN severity = 'High' THEN 'High Priority'
        WHEN severity = 'Medium' THEN 'Monitor'
        ELSE 'Low Priority'
    END AS priority_level
FROM alert
ORDER BY alert_id;
 alert_id | severity |    status     |  priority_level
----------+----------+---------------+------------------
        9 | High     | Investigating | High Priority
       10 | Critical | Investigating | Immediate Action
       11 | Medium   | Resolved      | Monitor
       12 | High     | Open          | High Priority
(4 rows)
```

Figure 23. VIEW – campaign threat summary

```
threat_intel_db=# SELECT *
FROM campaign_threat_summary
ORDER BY campaign_id;
 campaign_id |   campaign_name    | threat_actor  | indicator_count | avg_correlation
-------------+--------------------+---------------+-----------------+-----------------
           1 | Operation Shadow   | APT29         |               2 |            0.93
           2 | Operation Lazarus  | Lazarus Group |               1 |            0.88
           3 | Operation Carbanak | FIN7          |               2 |            0.82
(3 rows)
```

Figure 24. EXISTS – campaigns with indicators

```
threat_intel_db=# SELECT
    c.campaign_id,
    c.name AS campaign
FROM campaign c
WHERE EXISTS (
    SELECT 1
    FROM campaign_indicator ci
    WHERE ci.campaign_id = c.campaign_id
)
ORDER BY c.campaign_id;
 campaign_id |       campaign
-------------+----------------------
           1 | Operation Shadow
           2 | Operation Lazarus
           3 | Operation Carbanak
(3 rows)
```

Figure 25. NOT EXISTS – assets without resolved alerts

```
threat_intel_db=# SELECT
    a.asset_id,
    a.hostname,
    a.ip_address
FROM asset a
WHERE NOT EXISTS (
    SELECT 1
    FROM alert al
    WHERE al.asset_id = a.asset_id
      AND al.status = 'Resolved'
)
ORDER BY a.asset_id;
 asset_id |     hostname       |  ip_address
----------+--------------------+---------------
        1 | finance-server-01  | 192.168.1.10
        3 | backup-server-01   | 192.168.1.30
(2 rows)
```

3.5 Transactions, Indexes and Final Queries

Figure 26. Transaction – SAVEPOINT and COMMIT

```
threat_intel_db=# BEGIN;
BEGIN
threat_intel_db=*# UPDATE alert
SET status = 'Investigating'
WHERE alert_id = 2;
UPDATE 0
threat_intel_db=*# SAVEPOINT before_severity_change;
SAVEPOINT
threat_intel_db=*# UPDATE alert
SET severity = 'High'
WHERE alert_id = 2;
UPDATE 0
threat_intel_db=*# COMMIT;
COMMIT
threat_intel_db=# SELECT
    alert_id,
    severity,
    status
FROM alert
WHERE alert_id = 2;
 alert_id | severity | status
----------+----------+--------
(0 rows)
```

Figure 27. Transaction – full ROLLBACK

```
threat_intel_db=# BEGIN;
BEGIN
threat_intel_db=*# UPDATE campaign
SET status = 'Active'
WHERE campaign_id = 3;
UPDATE 1
threat_intel_db=*# SELECT campaign_id, name, status
FROM campaign
WHERE campaign_id = 3;
 campaign_id |        name         | status
-------------+---------------------+--------
           3 | Operation Carbanak | Active
(1 row)

threat_intel_db=*# ROLLBACK;
ROLLBACK
threat_intel_db=# SELECT campaign_id, name, status
FROM campaign
WHERE campaign_id = 3;
 campaign_id |        name         | status
-------------+---------------------+---------
           3 | Operation Carbanak | Dormant
(1 row)
```

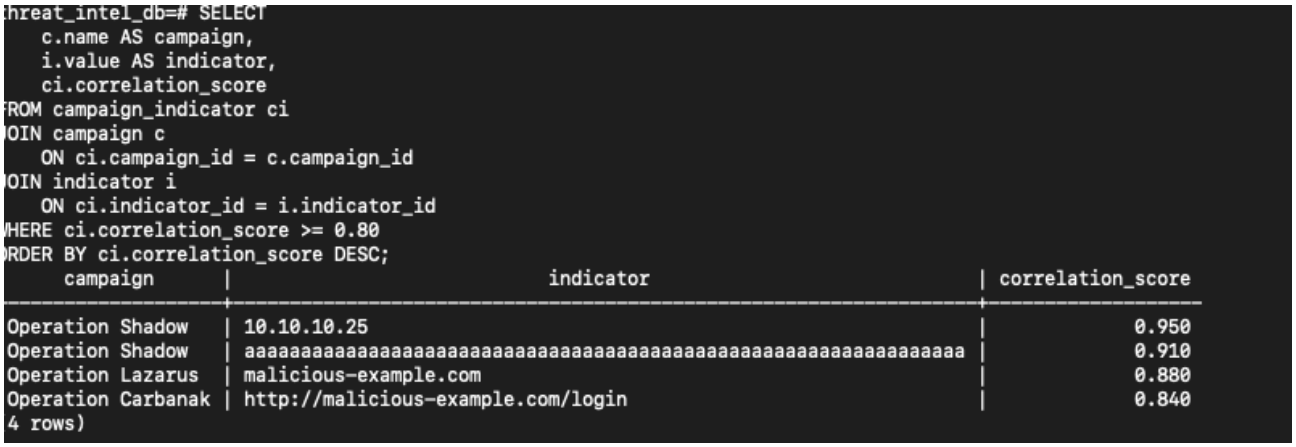
Figure 28. Indexes – index listing

```
threat_intel_db=# \di
                              List of relations
 Schema |              Name               | Type  | Owner |       Table
--------+---------------------------------+-------+-------+--------------------
 public | USER_email_key                  | index | amit  | USER
 public | USER_pkey                       | index | amit  | USER
 public | alert_pkey                      | index | amit  | alert
 public | asset_pkey                      | index | amit  | asset
 public | campaign_indicator_pkey         | index | amit  | campaign_indicator
 public | campaign_pkey                   | index | amit  | campaign
 public | campaign_ttp_pkey               | index | amit  | campaign_ttp
 public | idx_alert_severity              | index | amit  | alert
 public | idx_alert_status_severity       | index | amit  | alert
 public | idx_asset_org                   | index | amit  | asset
 public | idx_campaign_actor              | index | amit  | campaign
 public | idx_indicator_confidence        | index | amit  | indicator
 public | idx_indicator_source            | index | amit  | indicator
 public | idx_indicator_value             | index | amit  | indicator
 public | idx_malware_hash                | index | amit  | malware_sample
 public | idx_report_campaign             | index | amit  | report
 public | indicator_ioc_type_value_key    | index | amit  | indicator
 public | indicator_malware_pkey          | index | amit  | indicator_malware
 public | indicator_pkey                  | index | amit  | indicator
 public | intel_source_pkey               | index | amit  | intel_source
 public | malware_sample_pkey             | index | amit  | malware_sample
 public | malware_sample_sha256_hash_key  | index | amit  | malware_sample
 public | malware_vuln_pkey               | index | amit  | malware_vuln
 public | organization_pkey               | index | amit  | organization
 public | report_pkey                     | index | amit  | report
 public | threat_actor_pkey               | index | amit  | threat_actor
 public | ttp_mitre_technique_id_key      | index | amit  | ttp
 public | ttp_pkey                        | index | amit  | ttp
 public | vulnerability_pkey              | index | amit  | vulnerability
(29 rows)
```

Figure 29. High/Critical security alerts

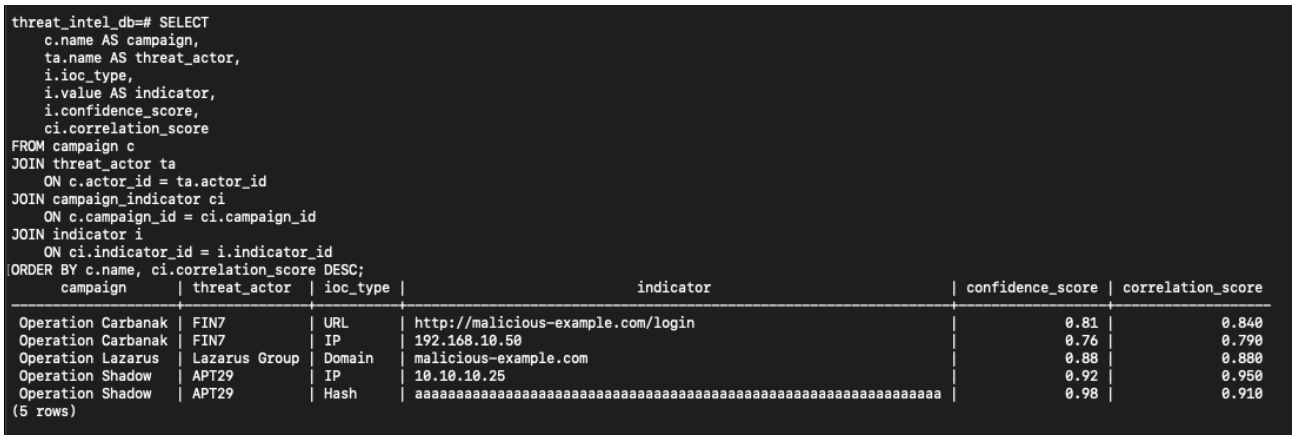
```
threat_intel_db=# SELECT
    al.alert_id,
    a.hostname,
    i.value AS indicator,
    al.severity,
    al.status
FROM alert al
JOIN asset a
    ON al.asset_id = a.asset_id
JOIN indicator i
    ON al.indicator_id = i.indicator_id
WHERE al.severity IN ('High', 'Critical')
ORDER BY
    CASE al.severity
        WHEN 'Critical' THEN 1
        WHEN 'High' THEN 2
        ELSE 3
    END;
 alert_id |     hostname      |                                                
```

Figure 30. Strong campaign correlations



```
threat_intel_db=# SELECT
    c.name AS campaign,
    i.value AS indicator,
    ci.correlation_score
FROM campaign_indicator ci
JOIN campaign c
    ON ci.campaign_id = c.campaign_id
JOIN indicator i
    ON ci.indicator_id = i.indicator_id
WHERE ci.correlation_score >= 0.80
ORDER BY ci.correlation_score DESC;
      campaign      |                                                                  indicator                                                                   | correlation_score
--------------------+--------------------------------------------------------------------------------------------------------------------------------------------+-------------------
 Operation Shadow   | 10.10.10.25                                                                                                                                |             0.950
 Operation Shadow   | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |             0.910
 Operation Lazarus  | malicious-example.com                                                                                                                      |             0.880
 Operation Carbanak | http://malicious-example.com/login                                                                                                         |             0.840
4 rows)
```

Figure 31. Final threat intelligence correlation query



```
threat_intel_db=# SELECT
    c.name AS campaign,
    ta.name AS threat_actor,
    i.ioc_type,
    i.value AS indicator,
    i.confidence_score,
    ci.correlation_score
FROM campaign c
JOIN threat_actor ta
    ON c.actor_id = ta.actor_id
JOIN campaign_indicator ci
    ON c.campaign_id = ci.campaign_id
JOIN indicator i
    ON ci.indicator_id = i.indicator_id
ORDER BY c.name, ci.correlation_score DESC;
      campaign      | threat_actor  | ioc_type |                                                                  indicator                                                                   | confidence_score | correlation_score
--------------------+---------------+----------+--------------------------------------------------------------------------------------------------------------------------------------------+------------------+-------------------
 Operation Carbanak | FIN7          | URL      | http://malicious-example.com/login                                                                                                         |             0.81 |             0.840
 Operation Carbanak | FIN7          | IP       | 192.168.10.50                                                                                                                              |             0.76 |             0.790
 Operation Lazarus  | Lazarus Group | Domain   | malicious-example.com                                                                                                                      |             0.88 |             0.880
 Operation Shadow   | APT29         | IP       | 10.10.10.25                                                                                                                                |             0.92 |             0.950
 Operation Shadow   | APT29         | Hash     | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa |             0.98 |             0.910
(5 rows)
```

4. Conclusion

The completed SQL implementation demonstrates the functioning of the project database and its major DBMS features. The executed queries show how threat actors, campaigns, indicators, malware, vulnerabilities, alerts and reports can be related and analyzed. The report also demonstrates advanced SQL features and transaction control required for the database implementation.

5. SQL Evidence Summary

Figures	Content
Figures 1–9	Project entity data and core tables
Figures 10–15	Relationship and multi-table JOIN queries
Figures 16–19	UPDATE and aggregate SQL
Figures 20–25	Subqueries, CASE, VIEW, EXISTS and NOT EXISTS
Figures 26–27	Transactions and rollback
Figure 28	Indexes
Figures 29–31	Final project-specific threat intelligence queries